

Logic Gate Implementation Using Single Layer Perceptron

Objective

To implement an logic gates using a Single Layer Perceptron and visualize:

- Perceptron architecture
- AND gate XOR gate classification
- Decision boundary
- Convergence of the perceptron (also the failure of convergence for XOR gate)
- Prediction for user-provided input

Perceptron Learning Rule

The prediction error is calculated as:

$$e = y - \hat{y}$$

The weights are updated using:

$$w_i^{new} = w_i^{old} + \eta e x_i$$

The bias is updated using:

$$b^{new} = b^{old} + \eta e$$

where η is the learning rate.

In this implementation:

$$\eta = 0.1$$

and the maximum number of epochs is:

$$100$$

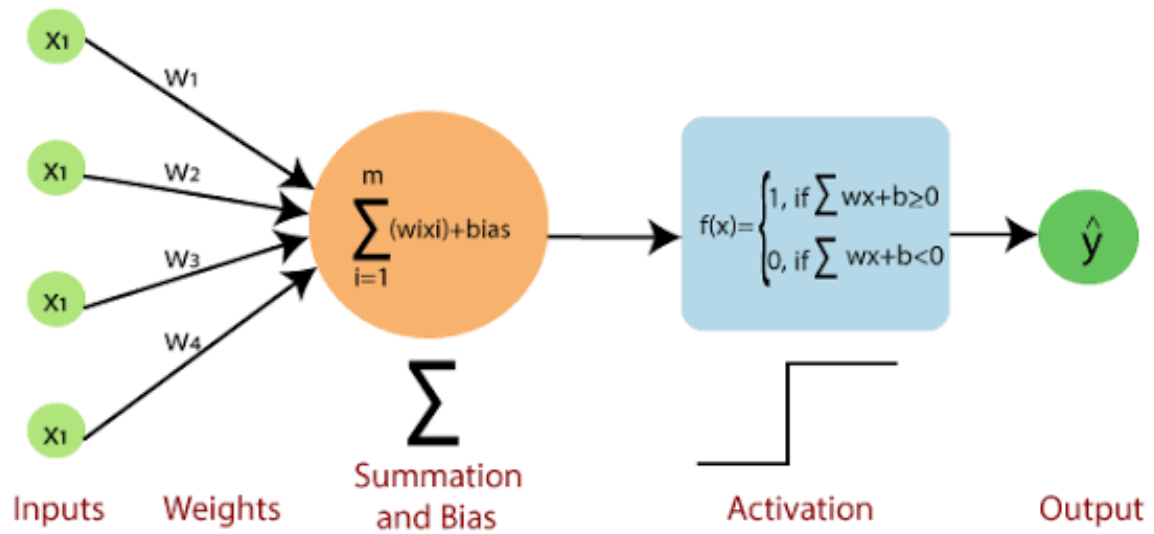


Figure 1: Perceptron Diagram

Perceptron Model

The weighted sum is calculated as

$$z = w_1 x_1 + w_2 x_2 + b$$

where:

- w_1 = weight associated with X_1
- w_2 = weight associated with X_2
- b = bias
- z = weighted sum

The Step activation function is:

$$f(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Part 1 : AND gate

0.1 Training Data

The AND gate has two binary inputs and one binary output.

X_1	X_2	Y
0	0	0
0	1	0
1	0	0
1	1	1

Table 1: AND Gate Truth Table

1 Python Implementation

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Circle, FancyArrowPatch
4
5 # =====
6 # 1. AND GATE TRAINING DATA
7 # =====
8
9 X = np.array([
10     [0, 0],
11     [0, 1],
12     [1, 0],
13     [1, 1]
14 ], dtype=float)
15
16 y = np.array([0, 0, 0, 1])
17
18
19 # =====
20 # 2. INITIALIZE WEIGHTS AND BIAS
21 # =====
22
23 weights = np.array([0.0, 0.0])
24 bias = 0.0
25
26 learning_rate = 0.1
27 max_epochs = 100
28
29
30 # =====
31 # 3. STEP ACTIVATION FUNCTION
32 # =====
33
34 def activation(z):
```

```

35
36     if z > 0:
37         return 1
38     else:
39         return 0
40
41
42 # =====
43 # 4. PERCEPTRON TRAINING
44 # =====
45
46 error_history = []
47
48 print("\n=====")
49 print("PERCEPTRON TRAINING")
50 print("=====")
51
52 for epoch in range(max_epochs):
53
54     errors = 0
55
56     for i in range(len(X)):
57
58         z = np.dot(X[i], weights) + bias
59
60         prediction = activation(z)
61
62         error = y[i] - prediction
63
64         if error != 0:
65
66             weights = weights + \
67                 learning_rate * error * X[i]
68
69             bias = bias + learning_rate * error
70
71             errors += 1
72
73     error_history.append(errors)
74
75     print(
76         f"Epoch {epoch + 1}: "
77         f"Weights = {weights}, "
78         f"Bias = {bias:.2f}, "
79         f"Errors = {errors}"
80     )
81
82     if errors == 0:
83
84         print("\nPerceptron converged!")
85

```

```

86         print(
87             f"Converged after {epoch + 1} epochs."
88         )
89
90         break
91
92     else:
93
94         print(
95             "\nPerceptron did not converge "
96             "within the maximum epochs."
97         )
98
99
100 # =====
101 # 5. FINAL PARAMETERS
102 # =====
103
104 print("\n=====")
105 print("FINAL PERCEPTRON PARAMETERS")
106 print("=====")
107
108 print("Weight 1:", weights[0])
109 print("Weight 2:", weights[1])
110 print("Bias:", bias)
111
112
113 # =====
114 # 6. USER INPUT
115 # =====
116
117 print("\n=====")
118 print("AND GATE PREDICTION")
119 print("=====")
120
121 while True:
122
123     try:
124
125         x1 = float(input("Enter X1 (0 or 1): "))
126         x2 = float(input("Enter X2 (0 or 1): "))
127
128         if x1 in [0, 1] and x2 in [0, 1]:
129             break
130
131         print("Please enter only 0 or 1.")
132
133     except ValueError:
134
135         print("Invalid input.")
136

```

```

137
138 # =====
139 # 7. USER PREDICTION
140 # =====
141
142 user_input = np.array([x1, x2])
143
144 z_user = np.dot(user_input, weights) + bias
145
146 user_prediction = activation(z_user)
147
148 print("\nWeighted Sum:", z_user)
149 print("Predicted Output:", user_prediction)
150
151
152 # =====
153 # 8. CREATE FIGURE
154 # =====
155
156 fig = plt.figure(figsize=(20, 8))
157
158
159 # =====
160 # 9. PERCEPTRON ARCHITECTURE
161 # =====
162
163 ax1 = fig.add_axes([0.02, 0.15, 0.28, 0.70])
164
165 ax1.set_xlim(0, 10)
166 ax1.set_ylim(0, 10)
167 ax1.axis("off")
168
169 ax1.set_title(
170     "Single Layer Perceptron",
171     fontsize=20,
172     fontweight="bold"
173 )
174
175 input_positions = [(2, 7), (2, 3)]
176 input_labels = ["X1", "X2"]
177
178 for (x, y_pos), label in zip(
179     input_positions, input_labels):
180
181     circle = Circle(
182         (x, y_pos),
183         0.7,
184         fill=False,
185         linewidth=3
186     )
187

```

```

188     ax1.add_patch(circle)
189
190     ax1.text(
191         x,
192         y_pos,
193         label,
194         ha="center",
195         va="center",
196         fontsize=18,
197         fontweight="bold"
198     )
199
200 ax1.text(
201     4.2,
202     7.3,
203     f"w1 = {weights[0]:.2f}",
204     fontsize=14
205 )
206
207 ax1.text(
208     4.2,
209     3.3,
210     f"w2 = {weights[1]:.2f}",
211     fontsize=14
212 )
213
214 neuron = Circle(
215     (6.5, 5),
216     1.1,
217     fill=False,
218     linewidth=3
219 )
220
221 ax1.add_patch(neuron)
222
223 ax1.text(
224     6.5,
225     5,
226     "Sigma + b\nStep",
227     ha="center",
228     va="center",
229     fontsize=15,
230     fontweight="bold"
231 )
232
233 output = Circle(
234     (9, 5),
235     0.7,
236     fill=False,
237     linewidth=3
238 )

```

```

239
240 ax1.add_patch(output)
241
242 ax1.text(
243     9,
244     5,
245     "Y",
246     ha="center",
247     va="center",
248     fontsize=18,
249     fontweight="bold"
250 )
251
252 arrow1 = FancyArrowPatch(
253     (2.7, 7),
254     (5.4, 5.5),
255     arrowstyle="->",
256     mutation_scale=20,
257     linewidth=2
258 )
259
260 arrow2 = FancyArrowPatch(
261     (2.7, 3),
262     (5.4, 4.5),
263     arrowstyle="->",
264     mutation_scale=20,
265     linewidth=2
266 )
267
268 arrow3 = FancyArrowPatch(
269     (7.6, 5),
270     (8.3, 5),
271     arrowstyle="->",
272     mutation_scale=20,
273     linewidth=2
274 )
275
276 ax1.add_patch(arrow1)
277 ax1.add_patch(arrow2)
278 ax1.add_patch(arrow3)
279
280 ax1.text(
281     6.5,
282     1.5,
283     f"Bias = {bias:.2f}",
284     ha="center",
285     fontsize=15,
286     fontweight="bold"
287 )
288
289

```

```

290 # =====
291 # 10. AND GATE CLASSIFICATION
292 # =====
293
294 ax2 = fig.add_axes([0.35, 0.15, 0.28, 0.70])
295
296 class_0 = X[y == 0]
297 class_1 = X[y == 1]
298
299 ax2.scatter(
300     class_0[:, 0],
301     class_0[:, 1],
302     s=180,
303     marker="o",
304     label="Output = 0",
305     zorder=3
306 )
307
308 ax2.scatter(
309     class_1[:, 0],
310     class_1[:, 1],
311     s=220,
312     marker="*",
313     label="Output = 1",
314     zorder=3
315 )
316
317
318 # =====
319 # 11. DECISION BOUNDARY
320 # =====
321
322 x_boundary = np.linspace(-0.2, 1.2, 100)
323
324 if abs(weights[1]) > 1e-10:
325
326     y_boundary = (
327         -(weights[0] * x_boundary + bias)
328         / weights[1]
329     )
330
331     ax2.plot(
332         x_boundary,
333         y_boundary,
334         linewidth=3,
335         label="Decision Boundary",
336         color="tab:green",
337         zorder=2
338     )
339
340 elif abs(weights[0]) > 1e-10:

```

```

341
342     x_vertical = -bias / weights[0]
343
344     ax2.axvline(
345         x=x_vertical,
346         linewidth=3,
347         label="Decision Boundary",
348         color="tab:green",
349         zorder=2
350     )
351
352
353 # =====
354 # 12. USER INPUT
355 # =====
356
357 ax2.scatter(
358     x1,
359     x2,
360     s=350,
361     facecolors="none",
362     edgecolors="red",
363     linewidths=2.5,
364     linestyle="--",
365     label=f"User Input ({x1:.0f},{x2:.0f}) -> "
366           f"{user_prediction}",
367     zorder=4
368 )
369
370 ax2.set_title(
371     "AND Gate Classification",
372     fontsize=20,
373     fontweight="bold"
374 )
375
376 ax2.set_xlabel("X1", fontsize=16)
377 ax2.set_ylabel("X2", fontsize=16)
378
379 ax2.set_xlim(-0.2, 1.2)
380 ax2.set_ylim(-0.2, 1.2)
381
382 ax2.set_xticks([0, 1])
383 ax2.set_yticks([0, 1])
384
385 ax2.grid(True, alpha=0.3)
386
387 ax2.legend(
388     fontsize=10,
389     loc="lower left"
390 )
391

```

```

392
393 # =====
394 # 13. CONVERGENCE GRAPH
395 # =====
396
397 ax3 = fig.add_axes([0.68, 0.15, 0.28, 0.70])
398
399 epochs = np.arange(
400     1,
401     len(error_history) + 1
402 )
403
404 ax3.plot(
405     epochs,
406     error_history,
407     marker="o",
408     linewidth=3,
409     markersize=8
410 )
411
412 ax3.set_title(
413     "Perceptron Convergence",
414     fontsize=20,
415     fontweight="bold"
416 )
417
418 ax3.set_xlabel(
419     "Epoch",
420     fontsize=16
421 )
422
423 ax3.set_ylabel(
424     "Number of Errors",
425     fontsize=16
426 )
427
428 ax3.set_xticks(epochs)
429
430 ax3.set_yticks(
431     range(
432         0,
433         max(error_history) + 1
434     )
435 )
436
437 ax3.grid(
438     True,
439     alpha=0.3
440 )
441
442

```

```

443 # =====
444 # 14. MARK CONVERGENCE POINT
445 # =====
446
447 if error_history[-1] == 0:
448
449     convergence_epoch = len(error_history)
450
451     ax3.scatter(
452         convergence_epoch,
453         0,
454         s=120,
455         zorder=5
456     )
457
458     ax3.annotate(
459         f"Converged at Epoch {convergence_epoch}",
460         xy=(convergence_epoch, 0),
461         xytext=(
462             max(1, convergence_epoch - 2),
463             max(error_history) * 0.6 + 0.5
464         ),
465         fontsize=13,
466         arrowprops=dict(
467             arrowstyle="->",
468             linewidth=2
469         )
470     )
471
472
473 # =====
474 # 15. MAIN TITLE
475 # =====
476
477 fig.suptitle(
478     "AND Gate - Single Layer Perceptron",
479     fontsize=25,
480     fontweight="bold"
481 )
482
483 plt.show()

```

1.1 Convergence

The perceptron is considered to have converged when an entire training epoch is completed with zero classification errors.

Therefore, the convergence condition is:

$$\text{Number of Errors} = 0$$

The convergence graph plots:

Epoch vs. Number of Errors

As training progresses, the number of errors decreases and eventually reaches zero.

1.2 Decision Boundary

The decision boundary of the perceptron is obtained from:

$$w_1x_1 + w_2x_2 + b = 0$$

Solving for x_2 :

$$x_2 = -\frac{w_1x_1 + b}{w_2}$$

This line separates the input points classified as output 0 from those classified as output 1.

1.3 User Input Prediction

The program allows the user to enter:

$$X_1 \in \{0, 1\}$$

and

$$X_2 \in \{0, 1\}$$

The weighted sum is calculated as:

$$z = w_1X_1 + w_2X_2 + b$$

The Step activation function then produces the final prediction.

1.4 Result

The Single Layer Perceptron successfully learns the AND gate because the AND gate is a **linearly separable Boolean function**.

The perceptron converges when the number of classification errors becomes zero.

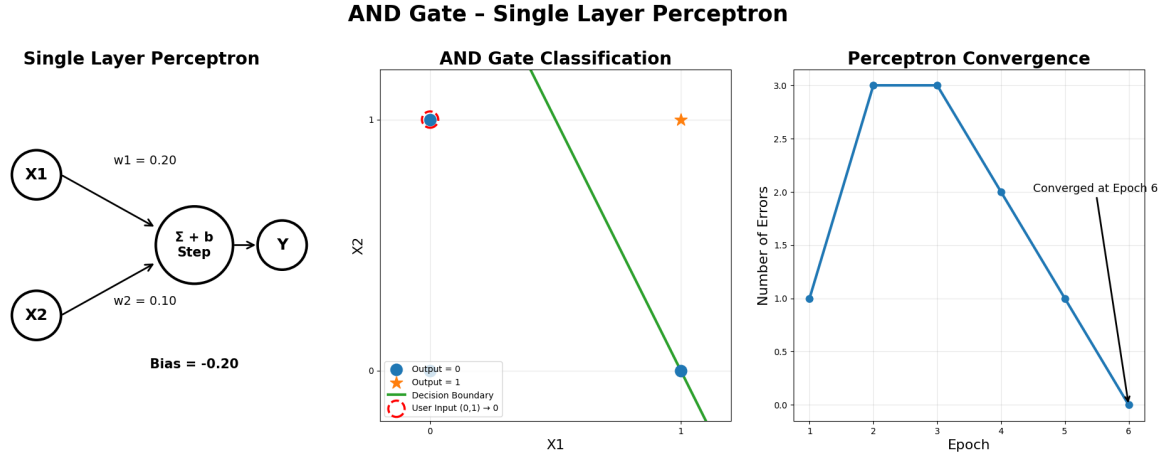


Figure 2: OUTPUT

The following table presents the training results of the single-layer perceptron for the AND gate.

Table 2: AND Gate Perceptron Training Results

Epoch	Weights	Bias	Errors
1	[0.1, 0.1]	0.10	1
2	[0.2, 0.1]	0.00	3
3	[0.2, 0.1]	-0.10	3
4	[0.2, 0.2]	-0.10	2
5	[0.2, 0.1]	-0.20	1
6	[0.2, 0.1]	-0.20	0

1.5 Convergence Analysis

The perceptron successfully converged after 6 epochs. The number of classification errors gradually decreased and reached zero in the sixth epoch.

Perceptron converged!
Converged after 6 epochs.

1.6 Final Perceptron Parameters

After convergence, the final parameters of the perceptron are:

Table 3: Final AND Gate Perceptron Parameters

Parameter	Value
Weight 1 (w_1)	0.2
Weight 2 (w_2)	0.1
Bias (b)	-0.2

The final perceptron equation is given by:

$$z = 0.2x_1 + 0.1x_2 - 0.2$$

The output is obtained using the step activation function:

$$\hat{y} = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

1.7 AND Gate Prediction

The program allows the user to provide an input for testing the trained perceptron.

===== AND GATE PREDICTION =====

Enter X1 (0 or 1): 0

Enter X2 (0 or 1): 1

Weighted Sum: -0.1

Predicted Output: 0

For the given input,

$$X = [0, 1]$$

the weighted sum is:

$$z = (0.2)(0) + (0.1)(1) - 0.2$$

$$z = -0.1$$

Since $z \leq 0$, the step activation function produces:

$$\boxed{\hat{y} = 0}$$

The predicted output is therefore consistent with the expected AND gate output for the input (0, 1).

1.8 Task

Try to perform perceptron on OR gate

2 XOR gate

The XOR gate produces an output of 1 when the two inputs are different and 0 when the two inputs are the same.

X_1	X_2	XOR Output Y
0	0	0
0	1	1
1	0	1
1	1	0

Table 4: XOR Gate Truth Table

2.1 Objective

To implement the XOR gate using a Single Layer Perceptron and demonstrate its non-convergence due to the non-linearly separable nature of the XOR problem.

The implementation also visualizes:

- Single Layer Perceptron architecture
- XOR gate classification
- Final decision boundary
- Non-convergence of the perceptron
- User input prediction

2.2 Training Data

The input matrix is:

$$X = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$$

The target vector is:

$$Y = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

2.3 Perceptron Model

The weighted sum is calculated as:

$$z = w_1x_1 + w_2x_2 + b$$

where w_1 and w_2 are the weights and b is the bias.

The Step activation function is:

$$f(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Therefore, the predicted output is:

$$\hat{y} = f(z)$$

2.4 Perceptron Learning Rule

The prediction error is:

$$e = y - \hat{y}$$

The weights are updated using:

$$w_i^{new} = w_i^{old} + \eta e x_i$$

The bias is updated using:

$$b^{new} = b^{old} + \eta e$$

where η is the learning rate.

In this implementation:

$$\eta = 0.1$$

and the maximum number of epochs is:

$$20$$

2.5 Python Implementation

```

1
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from matplotlib.patches import Circle, FancyArrowPatch
5
6
7 # =====
8 # 1. XOR GATE TRAINING DATA
9 # =====
10
11 X = np.array([
12     [0, 0],
13     [0, 1],
14     [1, 0],
15     [1, 1]
16 ])
17
18 y = np.array([0, 1, 1, 0])
19
20
21 # =====
22 # 2. INITIALIZE WEIGHTS AND BIAS
23 # =====
24

```

```

25 weights = np.array([0.0, 0.0])
26 bias = 0.0
27
28 learning_rate = 0.1
29 max_epochs = 20
30
31
32 # =====
33 # 3. STEP ACTIVATION FUNCTION
34 # =====
35
36 def activation(z):
37     return 1 if z > 0 else 0
38
39
40 # =====
41 # 4. PERCEPTRON TRAINING
42 # =====
43
44 error_history = []
45
46 for epoch in range(max_epochs):
47
48     errors = 0
49
50     for i in range(len(X)):
51
52         z = np.dot(X[i], weights) + bias
53
54         prediction = activation(z)
55
56         error = y[i] - prediction
57
58         if error != 0:
59
60             weights = (
61                 weights
62                 + learning_rate * error * X[i]
63             )
64
65             bias = (
66                 bias
67                 + learning_rate * error
68             )
69
70             errors += 1
71
72     error_history.append(errors)
73
74     print(
75         f"Epoch {epoch + 1}: "

```

```

76         f"Weights = {weights}, "
77         f"Bias = {bias:.2f}, "
78         f"Errors = {errors}"
79     )
80
81     if errors == 0:
82
83         print("\nPerceptron converged!")
84         break
85
86 else:
87
88     print(
89         "\nPerceptron did NOT converge "
90         "within the maximum epochs!"
91     )
92
93
94 # =====
95 # 5. USER INPUT
96 # =====
97
98 print("\n=====")
99 print("XOR GATE PREDICTION")
100 print("=====")
101
102 x1 = float(input("Enter X1 (0 or 1): "))
103 x2 = float(input("Enter X2 (0 or 1): "))
104
105 user_input = np.array([x1, x2])
106
107 z_user = (
108     np.dot(user_input, weights)
109     + bias
110 )
111
112 user_prediction = activation(z_user)
113
114
115 # =====
116 # 6. CREATE FIGURE
117 # =====
118
119 fig = plt.figure(figsize=(20, 8))
120
121
122 # =====
123 # 7. PERCEPTRON ARCHITECTURE DIAGRAM
124 # =====
125
126 ax1 = fig.add_axes([0.02, 0.15, 0.28, 0.7])

```

```

127
128 ax1.set_xlim(0, 10)
129 ax1.set_ylim(0, 10)
130 ax1.axis("off")
131
132 ax1.set_title(
133     "Single Layer Perceptron",
134     fontsize=20,
135     fontweight="bold"
136 )
137
138 input_positions = [
139     (2, 7),
140     (2, 3)
141 ]
142
143 input_labels = [
144     "X1",
145     "X2"
146 ]
147
148 for (x, y_pos), label in zip(
149     input_positions,
150     input_labels):
151
152     circle = Circle(
153         (x, y_pos),
154         0.7,
155         fill=False,
156         linewidth=3
157     )
158
159     ax1.add_patch(circle)
160
161     ax1.text(
162         x,
163         y_pos,
164         label,
165         ha="center",
166         va="center",
167         fontsize=18,
168         fontweight="bold"
169     )
170
171 ax1.text(
172     4.2,
173     7.3,
174     f"w1 = {weights[0]:.2f}",
175     fontsize=14
176 )
177

```

```

178 ax1.text(
179     4.2,
180     3.3,
181     f"w2 = {weights[1]:.2f}",
182     fontsize=14
183 )
184
185 neuron = Circle(
186     (6.5, 5),
187     1.1,
188     fill=False,
189     linewidth=3
190 )
191
192 ax1.add_patch(neuron)
193
194 ax1.text(
195     6.5,
196     5,
197     "    + b\nStep",
198     ha="center",
199     va="center",
200     fontsize=15,
201     fontweight="bold"
202 )
203
204 output = Circle(
205     (9, 5),
206     0.7,
207     fill=False,
208     linewidth=3
209 )
210
211 ax1.add_patch(output)
212
213 ax1.text(
214     9,
215     5,
216     "Y",
217     ha="center",
218     va="center",
219     fontsize=18,
220     fontweight="bold"
221 )
222
223 arrow1 = FancyArrowPatch(
224     (2.7, 7),
225     (5.4, 5.5),
226     arrowstyle="->",
227     mutation_scale=20,
228     linewidth=2

```

```

229 )
230
231 arrow2 = FancyArrowPatch(
232     (2.7, 3),
233     (5.4, 4.5),
234     arrowstyle="->",
235     mutation_scale=20,
236     linewidth=2
237 )
238
239 arrow3 = FancyArrowPatch(
240     (7.6, 5),
241     (8.3, 5),
242     arrowstyle="->",
243     mutation_scale=20,
244     linewidth=2
245 )
246
247 ax1.add_patch(arrow1)
248 ax1.add_patch(arrow2)
249 ax1.add_patch(arrow3)
250
251 ax1.text(
252     6.5,
253     1.5,
254     f"Bias = {bias:.2f}",
255     ha="center",
256     fontsize=15,
257     fontweight="bold"
258 )
259
260
261 # =====
262 # 8. XOR GATE CLASSIFICATION GRAPH
263 # =====
264
265 ax2 = fig.add_axes([0.35, 0.15, 0.28, 0.7])
266
267 class_0 = X[y == 0]
268 class_1 = X[y == 1]
269
270 ax2.scatter(
271     class_0[:, 0],
272     class_0[:, 1],
273     s=180,
274     marker="o",
275     label="Output = 0",
276     zorder=3
277 )
278
279 ax2.scatter(

```

```

280     class_1[:, 0],
281     class_1[:, 1],
282     s=220,
283     marker="*",
284     label="Output = 1",
285     zorder=3
286 )
287
288
289 # =====
290 # 9. FINAL DECISION BOUNDARY
291 # =====
292
293 x_boundary = np.linspace(
294     -0.2,
295     1.2,
296     100
297 )
298
299 if abs(weights[1]) > 1e-10:
300
301     y_boundary = (
302         -(weights[0] * x_boundary + bias)
303         / weights[1]
304     )
305
306     ax2.plot(
307         x_boundary,
308         y_boundary,
309         linewidth=3,
310         label="Final Line (Fails)",
311         color="tab:red",
312         zorder=2
313     )
314
315 elif abs(weights[0]) > 1e-10:
316
317     x_vert = -bias / weights[0]
318
319     ax2.axvline(
320         x=x_vert,
321         linewidth=3,
322         label="Final Line (Fails)",
323         color="tab:red",
324         zorder=2
325     )
326
327
328 # =====
329 # 10. MARK USER INPUT
330 # =====

```

```

331
332 ax2.scatter(
333     x1,
334     x2,
335     s=350,
336     facecolors="none",
337     edgecolors="red",
338     linewidths=2.5,
339     linestyle="--",
340     label=(
341         f"User Input "
342         f"({x1:.0f},{x2:.0f}) "
343         f"-> {user_prediction}"
344     ),
345     zorder=4
346 )
347
348
349 # =====
350 # 11. GRAPH FORMATTING
351 # =====
352
353 ax2.set_title(
354     "XOR Gate Classification",
355     fontsize=20,
356     fontweight="bold"
357 )
358
359 ax2.set_xlabel(
360     "X1",
361     fontsize=16
362 )
363
364 ax2.set_ylabel(
365     "X2",
366     fontsize=16
367 )
368
369 ax2.set_xlim(
370     -0.2,
371     1.2
372 )
373
374 ax2.set_ylim(
375     -0.2,
376     1.2
377 )
378
379 ax2.set_xticks([0, 1])
380 ax2.set_yticks([0, 1])
381

```

```

382 ax2.grid(
383     True,
384     alpha=0.3
385 )
386
387 ax2.legend(
388     fontsize=10,
389     loc="lower left"
390 )
391
392
393 # =====
394 # 12. CONVERGENCE GRAPH
395 # =====
396
397 ax3 = fig.add_axes([0.68, 0.15, 0.28, 0.7])
398
399 epochs = range(
400     1,
401     len(error_history) + 1
402 )
403
404 ax3.plot(
405     epochs,
406     error_history,
407     marker="o",
408     color="tab:red",
409     linewidth=3,
410     markersize=8
411 )
412
413 ax3.set_title(
414     "Non-Convergence (Oscillation)",
415     fontsize=20,
416     fontweight="bold"
417 )
418
419 ax3.set_xlabel(
420     "Epoch",
421     fontsize=16
422 )
423
424 ax3.set_ylabel(
425     "Number of Errors",
426     fontsize=16
427 )
428
429 ax3.set_xticks(
430     list(epochs)
431 )
432

```

```

433 ax3.set_ylim(
434     -0.2,
435     4.2
436 )
437
438 ax3.grid(
439     True,
440     alpha=0.3
441 )
442
443
444 # =====
445 # 13. ANNOTATE NON-CONVERGENCE
446 # =====
447
448 ax3.text(
449     max(epochs) / 2,
450     3.5,
451     "Infinite Loop of Errors\n"
452     "(Never Reaches 0)",
453     fontsize=13,
454     fontweight="bold",
455     color="tab:red",
456     ha="center",
457     bbox=dict(
458         boxstyle="round",
459         facecolor="white",
460         edgecolor="red"
461     )
462 )
463
464
465 # =====
466 # 14. MAIN TITLE
467 # =====
468
469 fig.suptitle(
470     "XOR Gate - Single Layer Perceptron "
471     "(Failure Case)",
472     fontsize=25,
473     fontweight="bold"
474 )
475
476 plt.show()

```

2.6 Why Does the Perceptron Fail for XOR?

The XOR problem is **not** linearly separable.

The two output classes are:

Class 0 : (0, 0), (1, 1)

Class 1 : (0, 1), (1, 0)

A single straight line cannot separate these two classes correctly.

A Single Layer Perceptron can only generate a linear decision boundary of the form:

$$w_1x_1 + w_2x_2 + b = 0$$

Since no single straight line can correctly separate the XOR classes, the perceptron repeatedly changes its weights and bias in an attempt to correct the classification errors.

2.7 Non-Convergence

For a linearly separable problem, the perceptron can eventually reach:

$$\text{Number of Errors} = 0$$

For XOR, this condition cannot be permanently achieved using a Single Layer Perceptron.

Therefore:

$$\text{Number of Errors} \neq 0$$

and the training process continues to oscillate between different weight and bias values.

The convergence graph therefore demonstrates the failure of the perceptron to reach a zero-error solution.

2.8 Decision Boundary

The decision boundary is given by:

$$w_1x_1 + w_2x_2 + b = 0$$

or:

$$x_2 = -\frac{w_1x_1 + b}{w_2}$$

This is a straight line. Since XOR requires a non-linear separation, the final line generated by the perceptron cannot correctly classify all four input points.

2.9 User Input Prediction

The program allows the user to provide values for X_1 and X_2 .

The weighted sum is calculated as:

$$z = w_1X_1 + w_2X_2 + b$$

The Step activation function then produces the predicted output:

$$\hat{Y} = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Because the Single Layer Perceptron cannot learn XOR completely, the final prediction is based on the weights obtained after the specified training epochs.

2.10 Conclusion

The XOR experiment demonstrates the fundamental limitation of a Single Layer Perceptron.

Unlike the AND gate, which is linearly separable, XOR is not linearly separable. Consequently, a Single Layer Perceptron cannot learn XOR correctly and cannot converge to a zero-error solution.

To solve the XOR problem, a **Multi-Layer Perceptron (MLP)** with at least one hidden layer is required.

XOR is not linearly separable

Single Layer Perceptron cannot solve XOR

MLP with a hidden layer can solve XOR

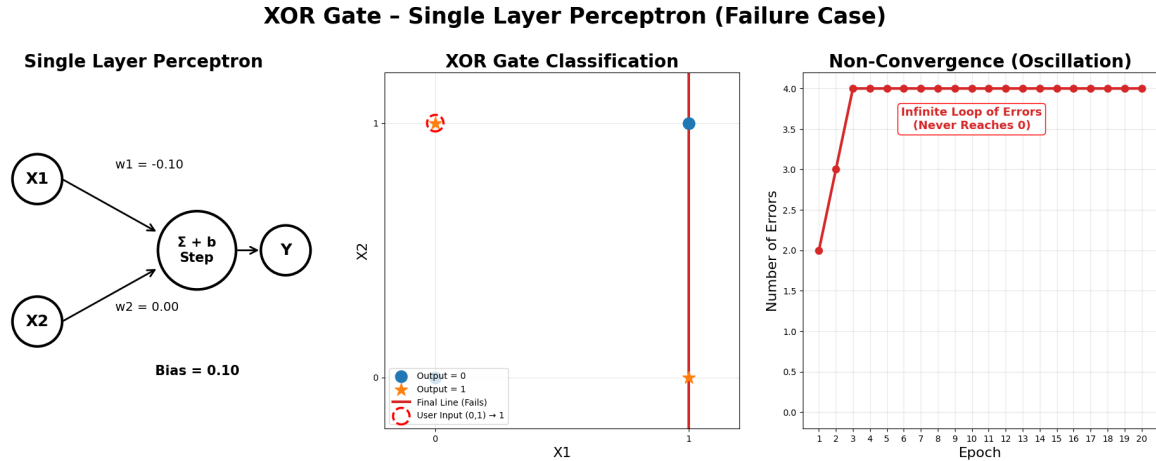


Figure 3: OUTPUT

2.11 Convergence Analysis

The perceptron did not converge within the maximum of 20 epochs. From Epoch 3 onward, the number of classification errors remains at 4, indicating that the model is unable to find a suitable linear decision boundary for the XOR problem.

Perceptron did NOT converge within the maximum epochs!

2.12 XOR Gate Prediction

The program also accepts user input for testing the trained perceptron.

===== XOR GATE PREDICTION =====

Enter X1 (0 or 1): 0

Enter X2 (0 or 1): 1

For the given input,

$$X = [0, 1]$$

the expected XOR output is

$$Y = 1$$

However, since the single-layer perceptron fails to learn the XOR decision boundary, its prediction depends on the final learned weights and bias and cannot reliably represent the XOR function.

2.13 Observation

The training results clearly demonstrate the limitation of a single-layer perceptron. Although the model updates its weights and bias, the error does not reach zero. Therefore, the XOR gate cannot be successfully implemented using a single-layer perceptron.

A **multi-layer perceptron (MLP)** with at least one hidden layer is required to correctly learn the XOR function.